



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

آزمایش هفتم: UART

نام درس

آزمایشگاه طراحی سیستم های دیجیتال

نیم سال ۱۴۰۴

استاد

دکتر انصاری

گروه ۸

امیرشایان سلیمانی نیا - ۴۰۳۱۰۶۱۰۸

حسن پناهی - ۴۰۳۱۰۵۸۳۳

امین استادکاظمی - ۴۰۳۱۷۰۸۹۳

فهرست مطالب

۱	مقدمه و هدف آزمایش	۲
۲	شرح آزمایش	۲
۳	ساختار مدار و پیاده‌سازی سخت‌افزاری	۳
۱.۳	تحلیل ماژول Transmitter	۳
۱.۱.۳	شماتیک مدار و تحلیل سخت‌افزاری	۳
۲.۱.۳	کد Verilog و تحلیل	۳
۲.۳	تحلیل ماژول Receiver	۴
۱.۲.۳	شماتیک مدار و تحلیل سخت‌افزاری	۴
۲.۲.۳	کد Verilog و تحلیل	۵
۳.۳	تحلیل ساختار Top-Level	۵
۱.۳.۳	شماتیک مدار و تحلیل سخت‌افزاری	۵
۲.۳.۳	کد Verilog و تحلیل	۶
۴	تست مدار و Waveform	۷
۱.۴	کد Verilog مربوط به Testbench	۷
۲.۴	خروجی‌ها، Waveform و تحلیل آن‌ها	۸
۵	نتیجه‌گیری	۹

۱ مقدمه و هدف آزمایش

هدف اصلی ما در این آزمایش، آشنایی با پروتکل‌های ارتباطی Asynchronous و طراحی یک سیستم UART با استفاده از زبان توصیف سخت‌افزار Verilog است. در این طراحی، انتقال داده بین دو واحد مجزای Transmitter و Receiver بر روی یک خط ارتباطی Serial شبیه‌سازی می‌شود.

در واقع هدف نهایی این است که با طراحی و پیاده‌سازی این دو بلوک و اتصال آن‌ها به یکدیگر در یک ماژول Top-Level، نشان دهیم که سیستم می‌تواند داده‌ها را به درستی پکیج کرده، روی خط انتقال قرار دهد و بدون خطا در سمت گیرنده واکشی کند.

۲ شرح آزمایش

برای پیاده‌سازی این ارتباط، باید استاندارد مشخصی را برای فریم‌های ارسالی رعایت کنیم. مدار Transmitter به گونه‌ای طراحی می‌شود که دیتای ۷ بیتی ASCII را دریافت کرده و با اضافه کردن بیت‌های کنترلی، آن را در قالب یک فریم ۱۰ بیتی ارسال می‌کند. این فریم به ترتیب شامل یک Start Bit، یک Parity Bit، ۷ بیت دیتای اصلی و در نهایت یک Stop Bit است.

در سمت دیگر، بلوک Receiver با تکیه بر یک State Machine به طور پیوسته وضعیت خط را رصد می‌کند. به محض تشخیص Start Bit، سیستم تغییر حالت داده و هشت بیت بعدی (داده اصلی و Parity Bit) را خوانده و درون یک Register هشت بیتی ذخیره می‌کند.

پورت‌های سیستم

در طراحی Top-Level، برای اعمال سناریوهای مختلف و کنترل مدار، پورت‌های زیر در نظر گرفته شده‌اند:

- **ورودی‌ها:** سیستم با سیگنال `clk` کار می‌کند و یک پورت `rst` برای Reset مدار دارد. برای فرمان شروع ارسال، یک سیگنال کنترلی به نام `start` تعبیه شده است. داده‌های ورودی نیز در قالب پورت ۷ بیتی `data_in` دریافت می‌شوند.
- **خروجی‌ها:** نتیجه نهایی ارتباط که همان دیتای واکشی شده در Receiver است، در قالب پورت ۸ بیتی `data_out` تولید می‌شود. برای اعلام پایان عملیات دریافت و معتبر بودن داده‌های خروجی، سیگنال `done` در نظر گرفته شده است که پس از دریافت کامل فریم، یک می‌شود.

۳ ساختار مدار و پیاده‌سازی سخت‌افزاری

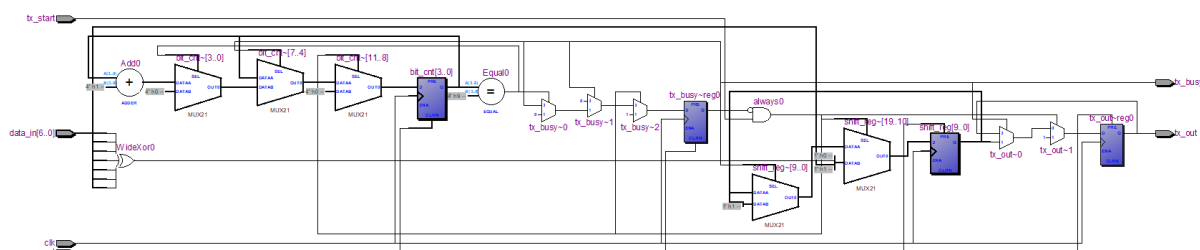
در این بخش، معماری داخلی سیستم طراحی شده را به تفکیک ماژول‌ها بررسی می‌کنیم. ساختار هر ماژول در دو بخش مجزای تحلیل سخت‌افزاری (بر اساس RTL) و تحلیل نرم‌افزاری (بر اساس کدهای Verilog) ارائه شده است.

۱.۳ تحلیل ماژول Transmitter

۱.۱.۳ شماتیک مدار و تحلیل سخت‌افزاری

ماژول Transmitter وظیفه دارد دیتای Parallel هفت بیتی را دریافت کرده و به یک جریان Serial ده بیتی تبدیل کند. در شماتیک RTL این بخش، یک گیت XOR چندورودی با نام WideXor0 دیده می‌شود که وظیفه تولید Parity Bit را بر عهده دارد. در مرکز مدار، مجموعه‌ای از Multiplexerها و یک Shift Register ده بیتی قرار دارند.

هنگامی که سیگنال tx_start فعال می‌شود، دیتا به صورت Parallel در این Register بارگذاری می‌شود. سپس در هر Clock، بیت‌ها به سمت راست Shift پیداکرده و LSB به خروجی tx_out منتقل می‌شود. یک Counter چهار بیتی با نام bit_cnt همراه با یک Comparator به نام Equal0 نیز در مدار وجود دارد که پس از رسیدن به عدد ۹، عملیات ارسال را متوقف کرده و سیگنال tx_busy را Low می‌کند.



شکل ۱: شماتیک RTL مربوط به ماژول Transmitter

۲.۱.۳ کد Verilog و تحلیل

در کد این ماژول، محاسبه Parity Bit با استفاده از عملگر کاهش data_in انجام می‌شود که تعداد یک‌ها را سنجیده و خروجی assign تولید می‌کند. در بلاک ترتیبی مدار، با فعال شدن شرط tx_start، فریم ده بیتی با استفاده از عملگر Concatenation ساخته می‌شود: مقدار ثابت ۱ برای انتهای فریم، دیتای اصلی، Parity Bit و در نهایت مقدار صفر برای Start Bit در جایگاه LSB قرار می‌گیرد. با ورود به وضعیت tx_busy، در هر کلاک داده‌ها با دستور shift_reg[9:1] به راست شیفت داده می‌شوند و بیت صفرم به پورت tx_out تخصیص می‌یابد تا جریان Serial شکل بگیرد.

```

assign parity = ^data_in;
always @(posedge clk or posedge rst) begin
  if (rst) begin
    tx_out <= 1'b1; tx_busy <= 1'b0; bit_cnt <= 4'd0; shift_reg <= 10'h3FF;
  end else begin
    if (tx_start && !tx_busy) begin
      shift_reg <= {1'b1, data_in, parity, 1'b0};
      tx_busy <= 1'b1; bit_cnt <= 4'd0; tx_out <= 1'b1;
    end else if (tx_busy) begin
      tx_out <= shift_reg[0];
      shift_reg <= {1'b1, shift_reg[9:1]};
      if (bit_cnt == 4'd9) begin
        tx_busy <= 1'b0; bit_cnt <= 4'd0;
      end else bit_cnt <= bit_cnt + 1'b1;
    end
  end
end
end

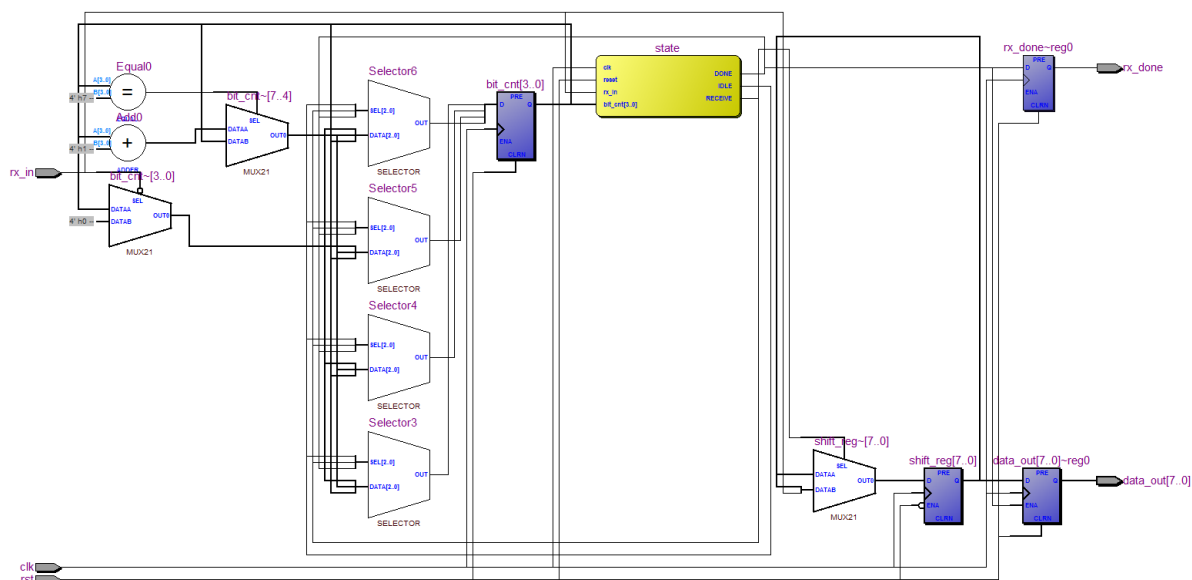
```

۲.۳ تحلیل ماژول Receiver

۱.۲.۳ شماتیک مدار و تحلیل سخت‌افزاری

ماژول Receiver بر اساس یک State Machine کنترل می‌شود که در شماتیک با یک بلوک زردرنگ مشخص شده است. در اینجا نیز از یک Counter چهار بیتی به نام bit_cnt برای شمارش ۸ بیت دریافتی (شامل دیتای اصلی و Parity Bit) استفاده شده است.

دیتا از طریق پورت rx_in وارد مدار شده و مسیر آن توسط Multiplexerها به سمت یک Shift Register هشت بیتی هدایت می‌شود. این مدار به گونه‌ای طراحی شده که Start Bit سیستم را تحریک می‌کند، اما فقط ۸ بیت پس از آن را ذخیره می‌کند. به محض اینکه Counter به عدد ۷ برسد، State Machine سیگنال rx_done را فعال کرده و محتوای کامل Register را روی data_out قرار می‌دهد.

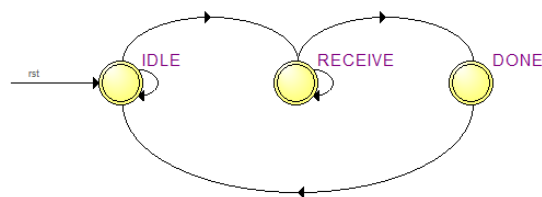


شکل ۲: شماتیک RTL مربوط به ماژول Receiver

تصاویر گراف و جدول زیر نیز نشان می‌دهند که سیستم با دریافت خط Low از وضعیت IDLE به RECEIVE رفته و پس از شمارش کامل، چرخه را در DONE به اتمام می‌رساند.

	Source State	Destination State	Condition
1	DONE	IDLE	
2	IDLE	RECEIVE	(rx_in)
3	IDLE	IDLE	(rx_in)
4	RECEIVE	RECEIVE	(bit_cnt[0]) + (bit_cnt[0]), (bit_cnt[1]) + (bit_cnt[1]), (bit_cnt[2]) + (bit_cnt[2]), (bit_cnt[3]) + (bit_cnt[3])
5	RECEIVE	DONE	(bit_cnt[0]), (bit_cnt[1]), (bit_cnt[2]), (bit_cnt[3])

شکل ۴: جدول Transition Conditions



شکل ۳: گراف State Machine

۲.۲.۳ کد Verilog و تحلیل

در ساختار نرم‌افزاری، State Machine با استفاده از یک ساختار case سه‌حالت پیاده‌سازی شده است. در بلاک IDLE، مدار پیوسته مقدار rx_in == 1'b0 را چک می‌کند تا Start Bit را شناسایی کند. با تایید این شرط، سیستم به RECEIVE می‌رود. در این مرحله، داده‌های سریال دریافتی با استفاده از عبارت {rx_in, shift_reg[7:1]} از سمت چپ وارد شده و سایر بیت‌ها به راست Shift پیدا می‌کنند. در همین حین، bit_cnt افزایش می‌یابد. زمانی که مقدار bit_cnt به عدد ۷ برسد (دریافت ۸ بیت کامل)، سیستم مستقیماً به DONE رفته، داده نهایی را روی پورت data_out تثبیت می‌کند و پالس rx_done را تولید می‌نماید.

```

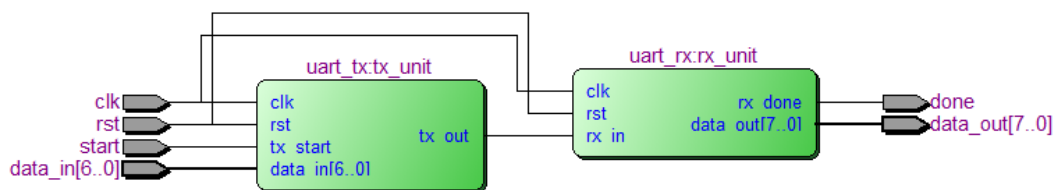
case (state)
  IDLE: begin
    if (rx_in == 1'b0) begin state <= RECEIVE; bit_cnt <= 4'd0; end
  end
  RECEIVE: begin
    shift_reg <= {rx_in, shift_reg[7:1]};
    if (bit_cnt == 4'd7) state <= DONE;
    else bit_cnt <= bit_cnt + 1'b1;
  end
  DONE: begin
    data_out <= shift_reg; rx_done <= 1'b1; state <= IDLE;
  end
  default: state <= IDLE;
endcase

```

۳.۳ تحلیل ساختار Top-Level

۱.۳.۳ شماتیک مدار و تحلیل سخت‌افزاری

برای تست یکپارچگی، ماژول‌های Transmitter و Receiver در یک بلوک Top-Level کپسوله شده‌اند. همان‌طور که در شماتیک RTL زیر مشاهده می‌شود، پورت‌های clk و rst به صورت مشترک به هر دو بلوک متصل شده‌اند تا همگام‌سازی پایه برقرار باشد. خروجی tx_out از طریق یک Wire میانی به نام serial_line مستقیماً به ورودی rx_in متصل شده است تا مسیر انتقال فیزیکی در داخل سیستم شبیه‌سازی شود.



شکل ۵: شماتیک RTL مربوط به Top-Level و اتصالات میانی

۲.۳.۳ کد Verilog و تحلیل

کد این بخش شامل تعریف متغیرهای ارتباطی و Instantiation ماژول‌های پایه‌ای است. متغیر serial_line از نوع wire تعریف شده و به عنوان پل ارتباطی بین خروجی فرستنده و ورودی گیرنده عمل می‌کند. با فراخوانی uart_tx و uart_rx و اتصال پورت‌های داخلی به پورت‌های Top-Level، سیستم نهایی ساختاری مشابه یک تراشه کامل پیدا می‌کند که داده‌های Parallel را دریافت، سریال‌سازی، ارسال و مجدداً به صورت Parallel واکشی می‌کند.

```
module uart_top (
    input clk, input rst, input start,
    input [6:0] data_in,
    output [7:0] data_out, output done
);
    wire serial_line, tx_busy;

    uart_tx tx_unit (.clk(clk), .rst(rst), .tx_start(start),
                    .data_in(data_in), .tx_out(serial_line), .tx_busy(tx_busy));

    uart_rx rx_unit (.clk(clk), .rst(rst), .rx_in(serial_line),
                    .data_out(data_out), .rx_done(done));
endmodule
```

۴ تست مدار و Waveform

۱.۴ کد Verilog مربوط به Testbench

```
`timescale 1ns/1ns

module tb_uart_top;

    reg clk;
    reg rst;
    reg start;
    reg [6:0] data_in;

    wire [7:0] data_out;
    wire done;

    reg [7:0] expected_val;
    integer pass_count = 0;
    integer fail_count = 0;

    uart_top uut (
        .clk(clk),
        .rst(rst),
        .start(start),
        .data_in(data_in),
        .data_out(data_out),
        .done(done)
    );

    always #5 clk = ~clk;

    task test_scenario(input [6:0] test_data, input integer test_num);
    begin
        data_in = test_data;
        expected_val = {test_data, ^test_data};

        start = 1;
        #10 start = 0;

        wait(done == 1);
        #10;

        if (data_out == expected_val) begin
            $display("Scenario%0d: Input=%b Expected=%b Received=%b->[PASS]", test_num, data_in, expected_val
, data_out);
            pass_count = pass_count + 1;
        end else begin
            $display("Scenario%0d: Input=%b Expected=%b Received=%b->[FAIL]", test_num, data_in, expected_val
, data_out);
            fail_count = fail_count + 1;
        end

        #50;
    end
endtask

initial begin
    clk = 0;
    rst = 1;
    start = 0;
    data_in = 7'b0000000;

    #20 rst = 0;
    #10;

    $display("UUUUUUUUUUUART Simulation Started...UUUUUUUUUUUUUUUUUUUUUUUUUUUUUU");

    test_scenario(7'b0000000, 1);
    test_scenario(7'b1111111, 2);
    test_scenario(7'b1010101, 3);
    test_scenario(7'b0101010, 4);
    test_scenario(7'b1100110, 5);
    test_scenario(7'b0011001, 6);
    test_scenario(7'b0001000, 7);

    $display("Simulation Finished!");
    $stop;
end
endmodule
```

۲.۴ خروجی‌ها، Waveform و تحلیل آن‌ها

پس از اجرای Testbench در ModelSim، لاگ‌های مربوط به ۷ سناریوی مختلف در ترمینال چاپ شد. در این کد از یک Task استفاده شده است که متغیر expected_val را با وصل کردن Parity Bit به انتهای داده اصلی تولید می‌کند تا مقایسه خروجی به صورت خودکار انجام شود.

```
#          UART Simulation Started...
# Scenario 1: Input = 0000000 | Expected = 00000000 | Received = 00000000 -> [PASS]
# Scenario 2: Input = 1111111 | Expected = 11111111 | Received = 11111111 -> [PASS]
# Scenario 3: Input = 1010101 | Expected = 10101010 | Received = 10101010 -> [PASS]
# Scenario 4: Input = 0101010 | Expected = 01010101 | Received = 01010101 -> [PASS]
# Scenario 5: Input = 1100110 | Expected = 11001100 | Received = 11001100 -> [PASS]
# Scenario 6: Input = 0011001 | Expected = 00110011 | Received = 00110011 -> [PASS]
# Scenario 7: Input = 0001000 | Expected = 00010001 | Received = 00010001 -> [PASS]
# Simulation Finished!
```

شکل ۶: لاگ‌های چاپ شده در ترمینال شبیه‌ساز شامل وضعیت ۷ سناریوی تست

با بررسی دقیق Waveform در بازه‌های زمانی مختلف، منطق عملکرد سیستم به شرح زیر تحلیل می‌شود:

- **سناریوی ۱ (الگوی 0000000):** در این حالت، دیتای ورودی شامل هیچ بیتی با ارزش یک نیست (زوج). بنابراین خروجی بلوک تولید Parity صفر است و فریم هدف به صورت 00000000 درمی‌آید. در Waveform مشاهده می‌شود که پس از اتمام شیفت‌ها، سیگنال done فعال شده و رجیستر خروجی آپدیت می‌شود.
- **سناریوی ۲ (الگوی 1111111):** در اینجا تعداد یک‌ها فرد است (هفت عدد)، بنابراین Parity Bit یک محاسبه شده و متغیر expected_val مقدار 1111111 را به خود می‌گیرد. پورت data_out نیز دقیقاً روی همین الگو تثبیت می‌شود.
- **سناریوی ۳ (الگوی 1010101):** این الگو دارای چهار بیت یک (زوج) است، پس Parity برابر صفر شده و مقدار مورد انتظار 10101010 خواهد بود. سیستم به درستی نوسانات خط را دنبال کرده و داده را واکنشی می‌کند.
- **سناریوی ۴ (الگوی 0101010):** برعکس حالت قبل، این الگو دارای سه بیت یک (فرد) است که منجر به تولید Parity یک و خروجی 01010101 می‌شود. State Machine بدون از دست دادن هماهنگی، این فریم را نیز به درستی دریافت می‌کند.
- **سناریوی ۵ (الگوی 1100110):** این الگو با چهار بیت یک، Parity صفر تولید می‌کند و انتظار می‌رود خروجی 11001100 باشد. پالس done در زمان‌بندی دقیق و مشابه سناریوهای قبلی فعال می‌شود.
- **سناریوی ۶ (الگوی 0011001):** با سه بیت یک در ورودی، Parity یک محاسبه شده و داده 00110011 روی پورت خروجی قرار می‌گیرد. متغیر pass_count با موفقیت به عدد ۶ می‌رسد.

